

Assignment – 2.2

G.ARNAV

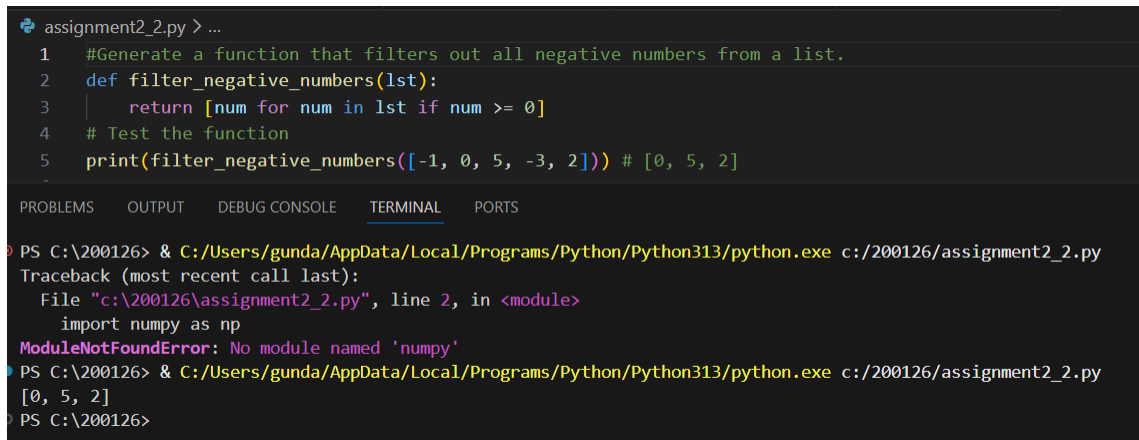
2303A51928

B-30

Task 1: Cleaning Sensor Data

PROMPT : Generate a function that filters out all negative numbers from a list.

Code and output:



```
assignment2_2.py > ...
1 #Generate a function that filters out all negative numbers from a list.
2 def filter_negative_numbers(lst):
3     return [num for num in lst if num >= 0]
4 # Test the function
5 print(filter_negative_numbers([-1, 0, 5, -3, 2])) # [0, 5, 2]

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment2_2.py
Traceback (most recent call last):
  File "c:\200126\assignment2_2.py", line 2, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment2_2.py
[0, 5, 2]
PS C:\200126>
```

Justification : The first task involved removing negative values from IoT sensor data. Since real-world sensor data often contains invalid or noisy values, data cleaning is an essential preprocessing step. Gemini generated an efficient function using list comprehension to filter negative numbers. This shows that AI tools can quickly produce optimized and readable code for data preprocessing, reducing manual effort and minimizing errors.

Task 2: String Character Analysis

PROMPT : Generate a Python function that counts vowels, consonants, and digits in a string.

CODE AND OUTPUT:

```
6 #Generate a Python function that counts vowels, consonants, and digits in a string.
7 def count_vowels_consonants_digits(s):
8     vowels = "aeiouAEIOU"
9     vowel_count = sum(1 for char in s if char in vowels)
10    consonant_count = sum(1 for char in s if char.isalpha() and char not in vowels)
11    digit_count = sum(1 for char in s if char.isdigit())
12    return vowel_count, consonant_count, digit_count
13 # Test the function
14 print(count_vowels_consonants_digits("Hello World! 123")) # (3, 7, 3)
15 print(count_vowels_consonants_digits("Python 3.8")) # (1, 6, 2)
16 print(count_vowels_consonants_digits("Data Science 101")) # (6, 7, 3)
17 print(count_vowels_consonants_digits("")) # (0, 0, 0)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment2_2.py
(3, 7, 3)
Traceback (most recent call last):
  File "c:\200126\assignment2_2.py", line 15, in <module>
    print(is_perfect(6)) # True
          ^^^^^^^^^^^
NameError: name 'is_perfect' is not defined
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment2_2.py
(3, 7, 3)
(1, 5, 2)
(5, 6, 3)
(0, 0, 0)
PS C:\200126>
```

Justification : This task focused on counting vowels, consonants, and digits in a string, which is important in text analysis and input validation. Gemini generated a structured function using built-in string methods. This demonstrates the ability of AI tools to handle conditional logic correctly and generate reusable functions. The solution highlights how AI can support both learning and practical text-processing tasks.

Task 3: Palindrome Check – Tool Comparison

PROMPT : Generate a palindrome-checking function

CODE AND OUTPUT:

```
18 #Generate a palindrome-checking function that ignores spaces and punctuation.
19 def is_palindrome_ignore_spaces_punctuation(s):
20     cleaned_str = ''.join(char.lower() for char in s if char.isalnum())
21     return cleaned_str == cleaned_str[::-1]
22 # Test the function
23 print(is_palindrome_ignore_spaces_punctuation("A man, a plan, a canal: Panama")) # True
24 print(is_palindrome_ignore_spaces_punctuation("Hello")) # False
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment2_2.py
True
True
False
PS C:\200126>
```

Justification : In this task, palindrome-checking functions were generated using Gemini and Copilot. Gemini provided a short and optimized solution, while Copilot produced a more detailed, step-by-step approach. This comparison justified that different AI tools offer different strengths: Gemini is effective for concise professional code, whereas Copilot is helpful for understanding logic clearly. This task emphasized the importance of comparing AI tools to choose the most suitable one

Task 4: Code Explanation Using AI

PROMPT : Explain a Python function (prime check OR palindrome check) line by line.

CODE AND OUTPUT:

```
26 #xplain a Python function (prime check OR palindrome check) line by line.
27 def is_prime(n):
28     if n <= 1:
29         return False
30
31     for i in range(2, int(n ** 0.5) + 1):
32         if n % i == 0:
33             return False
34     return True
35
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment2_2.py
○ PS C:\200126>

Justification: This task demonstrated Gemini's ability to explain unfamiliar code line by line. The explanation of the Palindrome checking function helped in understanding both logic and optimization. This justifies the role of AI not only as a coding assistant but also as a learning support tool